

```

import { useState, useEffect, useCallback, useRef } from "react";

/* — CONFIG — */
const VOTERS = ["Sarah", "Maria", "Erik", "Ryan", "Sam", "Mads", "Connor", "Josh"]
const VOTER_COLORS = ["#FF6B9D", "#FFD93D", "#6BCB77", "#4D96FF", "#FF9F45", "#C7",
const VOTER_EMOJIS = ["🌸", "✨", "🌿", "🌊", "🔥", "💜", "⚡", "🎯"];

// Eurovision 2026 Grand Final – Vienna, 16 May 2026
const ACTS = [
  { id: 1, country: "🇩🇰", name: "Denmark", act: "Søren Torpegaard Lund",
  { id: 2, country: "🇩🇪", name: "Germany", act: "Sarah Engels",
  { id: 3, country: "🇮🇱", name: "Israel", act: "Noam Bettan",
  { id: 4, country: "🇧🇪", name: "Belgium", act: "ESSYLA",
  { id: 5, country: "🇦🇱", name: "Albania", act: "Alis",
  { id: 6, country: "🇬🇷", name: "Greece", act: "Akylas",
  { id: 7, country: "🇺🇦", name: "Ukraine", act: "LELEKA",
  { id: 8, country: "🇦🇺", name: "Australia", act: "Delta Goodrem",
  { id: 9, country: "🇷🇸", name: "Serbia", act: "LAVINA",
  { id: 10, country: "🇲🇹", name: "Malta", act: "AIDAN",
  { id: 11, country: "🇨🇪", name: "Czechia", act: "Daniel Zizka",
  { id: 12, country: "🇧🇬", name: "Bulgaria", act: "DARA",
  { id: 13, country: "🇭🇷", name: "Croatia", act: "LELEK",
  { id: 14, country: "🇬🇧", name: "United Kingdom", act: "LOOK MUM NO COMPUTER",
  { id: 15, country: "🇫🇷", name: "France", act: "Monroe",
  { id: 16, country: "🇲🇩", name: "Moldova", act: "Satoshi",
  { id: 17, country: "🇫🇮", name: "Finland", act: "Linda Lampenius x Pete
  { id: 18, country: "🇵🇱", name: "Poland", act: "ALICJA",
  { id: 19, country: "🇱🇮", name: "Lithuania", act: "Lion Ceccah",
  { id: 20, country: "🇸🇪", name: "Sweden", act: "FELICIA",
  { id: 21, country: "🇨🇾", name: "Cyprus", act: "Antigoni",
  { id: 22, country: "🇮🇹", name: "Italy", act: "Sal Da Vinci",
  { id: 23, country: "🇳🇴", name: "Norway", act: "JONAS LOVV",
  { id: 24, country: "🇷🇴", name: "Romania", act: "Alexandra Căpitănescu",
  { id: 25, country: "🇦🇹", name: "Austria", act: "COSMÓ",
];

const RANK_MEDALS = ["🥇", "🥈", "🥉", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", '

/* — GOOGLE SHEETS HELPERS — */
// Sheet layout: Row 1 = headers, Rows 2-26 = one per act
// Columns: ActID | Country | Act | Song | Alex | Billie | Carlos | Dana | Emi |
const SHEET_HEADERS = ["ActID", "Country", "Act", "Song", ...VOTERS, "TotalScore"];

async function sheetsRequest(apiKey, sheetId, method, range, body) {

```

```

const base = `https://sheets.googleapis.com/v4/spreadsheets/${sheetId}/values`;
const url = method === "GET"
  ? `${base}/${encodeURIComponent(range)}?key=${apiKey}`
  : `${base}/${encodeURIComponent(range)}?valueInputOption=RAW&key=${apiKey}`;
const res = await fetch(url, {
  method,
  headers: { "Content-Type": "application/json" },
  ...(body ? { body: JSON.stringify(body) } : {}),
});
if (!res.ok) {
  const err = await res.json().catch(() => ({}));
  throw new Error(err?.error?.message || `HTTP ${res.status}`);
}
return res.json();
}

async function initSheet(apiKey, sheetId) {
  // Write headers + all acts on first use
  const rows = [SHEET_HEADERS];
  for (const act of ACTS) {
    rows.push([act.id, act.name, act.act, act.song, ...VOTERS.map(() => 0), 0]);
  }
  await sheetsRequest(apiKey, sheetId, "PUT", "Sheet1!A1:K26", { values: rows });
}

async function loadFromSheet(apiKey, sheetId) {
  const data = await sheetsRequest(apiKey, sheetId, "GET", "Sheet1!A1:K26");
  const rows = data.values || [];
  if (rows.length < 2) return null;
  // rows[0] = headers, rows[1..] = acts
  const ratings = {};
  for (let vi = 0; vi < VOTERS.length; vi++) ratings[vi] = {};
  for (let r = 1; r < rows.length; r++) {
    const row = rows[r];
    const actId = parseInt(row[0]);
    if (!actId) continue;
    for (let vi = 0; vi < VOTERS.length; vi++) {
      const val = parseInt(row[4 + vi]) || 0;
      if (val > 0) ratings[vi][actId] = val;
    }
  }
  return ratings;
}

async function saveToSheet(apiKey, sheetId, ratings, computeScore) {
  // Build data rows (skip header row 1, start at row 2)
  const updates = ACTS.map((act, i) => {

```

```

    const voterCols = VOTERS.map((_, vi) => ratings[vi]?.[act.id] ?? 0);
    const total = computeScore(act.id, ratings);
    return [act.id, act.name, act.act, act.song, ...voterCols, total];
  });
  await sheetsRequest(apiKey, sheetId, "PUT", "Sheet1!A2:K26", { values: updates
}

/* — STAR RATING ————— */
function Stars({ actId, voterId, ratings, onRate, color }) {
  const cur = ratings[voterId]?.[actId] ?? 0;
  return (
    <div style={{ display: "flex", gap: 1 }}>
      {[1,2,3,4,5].map(s => (
        <button key={s} onClick={() => onRate(voterId, actId, s === cur ? 0 : s)}
          background: "none", border: "none", cursor: "pointer",
          fontSize: 15, padding: "1px", lineHeight: 1,
          opacity: s <= cur ? 1 : 0.2,
          filter: s <= cur ? `drop-shadow(0 0 4px ${color})` : "none",
          transform: s <= cur ? "scale(1.05)" : "scale(1)",
          transition: "all 0.12s",
        >>★</button>
      ))}
    </div>
  );
}

/* — ACT CARD ————— */
function ActCard({ act, ratings, onRate, activeVoter, score, rank, total }) {
  const pct = total > 1 ? ((total - rank) / (total - 1)) * 100 : 0;
  const hasAnyRating = VOTERS.some((_, vi) => (ratings[vi]?.[act.id] ?? 0) > 0);
  return (
    <div style={{
      background: "rgba(255,255,255,0.025)",
      border: "1px solid rgba(255,255,255,0.07)",
      borderRadius: 14,
      padding: "14px 16px 16px",
      position: "relative",
      overflow: "hidden",
      transition: "border-color 0.3s",
      ...(rank < 3 && hasAnyRating ? { borderColor: ["rgba(255,215,0,0.35)", "rgba
    }}>
    {/* Score bar */}
    <div style={{
      position: "absolute", bottom: 0, left: 0, height: 2,
      width: `${pct}%`,
      background: `linear-gradient(90deg, ${VOTER_COLORS[activeVoter]}, #FFD93D
      transition: "width 0.6s cubic-bezier(0.4,0,0.2,1)",

```

```

        borderRadius: "0 1px 0 0",
    }} />

<div style={{ display: "flex", gap: 12, alignItems: "flex-start", marginBot
  <span style={{ fontSize: 32, lineHeight: 1, flexShrink: 0 }}>{act.country}
  <div style={{ flex: 1, minWidth: 0 }}>
    <div style={{ fontFamily: "'Barlow Condensed', sans-serif", fontWeight:
      {act.name}
    </div>
    <div style={{ fontSize: 11.5, color: "rgba(255,255,255,0.45)", marginTo
      {act.act} · <em>{act.song}</em>
    </div>
  </div>
  <div style={{ textAlign: "right", flexShrink: 0 }}>
    {score > 0 && <div style={{ fontFamily: "'Barlow Condensed', sans-serif
      {score > 0 && <div style={{ fontSize: 10, color: "rgba(255,255,255,0.3)
    </div>
  </div>

<div style={{ display: "grid", gridTemplateColumns: "repeat(3, 1fr)", gap:
  {VOTERS.map((voter, vi) => (
    <div key={vi} style={{
      background: vi === activeVoter ? `${VOTER_COLORS[vi]}18` : "rgba(0,0,
      border: `1px solid ${vi === activeVoter ? VOTER_COLORS[vi] + "55" : "
      borderRadius: 8, padding: "5px 8px",
      transition: "all 0.2s",
    }}>
    <div style={{ fontSize: 10, color: VOTER_COLORS[vi], fontWeight: 700,
      {VOTER_EMOJIS[vi]} {voter}
    </div>
    <Stars actId={act.id} voterId={vi} ratings={ratings} onRate={onRate}
    </div>
  )))
  </div>
</div>
);
}

/* — SHEET SETUP MODAL ————— */
function SheetModal({ onConnect, onClose }) {
  const [apiKey, setApiKey] = useState("");
  const [sheetId, setSheetId] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const handleConnect = async () => {
    if (!apiKey.trim() || !sheetId.trim()) { setError("Both fields required."); r

```

```

    setLoading(true); setError("");
    try {
      await onConnect(apiKey.trim(), sheetId.trim());
    } catch (e) {
      setError(e.message || "Connection failed. Check your API key and Sheet ID."
    } finally { setLoading(false); }
  };
};

return (
  <div style={{
    position: "fixed", inset: 0, zIndex: 100,
    background: "rgba(0,0,0,0.85)", backdropFilter: "blur(8px)",
    display: "flex", alignItems: "center", justifyContent: "center", padding: 2
  }}>
    <div style={{
      background: "#12121e", border: "1px solid rgba(255,255,255,0.12)",
      borderRadius: 18, padding: "28px 28px 24px", maxWidth: 480, width: "100%"
      boxShadow: "0 24px 80px rgba(0,0,0,0.6)",
    }}>
      <div style={{ display: "flex", justifyContent: "space-between", alignItem
        <div>
          <h2 style={{ margin: 0, fontFamily: "'Barlow Condensed', sans-serif",
            <img alt="Google Sheets logo" data-bbox="258 468 278 483"/> Connect Google Sheets
          </h2>
          <p style={{ margin: "6px 0 0", color: "rgba(255,255,255,0.45)", fontS
            Sync ratings live across all devices
          </p>
        </div>
        <button onClick={onClose} style={{ background: "none", border: "none",
      </div>

      <div style={{ background: "rgba(77,150,255,0.08)", border: "1px solid rgb
        <strong style={{ color: "#4D96FF" }}>Setup steps:</strong><br/>
        1. Create a new Google Sheet<br/>
        2. Share it → <em>Anyone with the link can edit</em><br/>
        3. Enable the <strong>Google Sheets API</strong> in <a href="https://co
        4. Create an <strong>API Key</strong> (no OAuth needed for public sheet
        5. Copy the Sheet ID from its URL: <code style={{ background: "rgba(255
      </div>

      {[
        { label: "Google Sheets API Key", value: apiKey, set: setApiKey, placeh
        { label: "Sheet ID", value: sheetId, set: setSheetId, placeholder: "1Bx
      ]}.map(({ label, value, set, placeholder, type }) => (
        <div key={label} style={{ marginBottom: 14 }}>
          <label style={{ display: "block", fontSize: 12, color: "rgba(255,255,
          <input

```

```

        type={type} value={value} onChange={e => set(e.target.value)}
        placeholder={placeholder}
        style={{
            width: "100%", background: "rgba(255,255,255,0.06)",
            border: "1px solid rgba(255,255,255,0.12)", borderRadius: 9,
            color: "#fff", padding: "10px 13px", fontSize: 13, outline: "none",
            boxSizing: "border-box",
        }}
    />
</div>
)))

{error && <div style={{ background: "rgba(255,80,80,0.1)", border: "1px s

<button
    onClick={handleConnect}
    disabled={loading}
    style={{
        width: "100%", padding: "12px",
        background: loading ? "rgba(77,150,255,0.3)" : "rgba(77,150,255,0.2)",
        border: "1px solid rgba(77,150,255,0.5)", borderRadius: 10,
        color: "#4D96FF", fontWeight: 800, fontSize: 15, cursor: loading ? "n
        fontFamily: "'Barlow Condensed', sans-serif", letterSpacing: 1,
        transition: "all 0.2s",
    }}
>
    {loading ? "Connecting..." : "🔌 Connect & Initialise Sheet"}
</button>
</div>
</div>
);
}

/* ——— MAIN APP ——— */
export default function EurovisionRanker() {
    const [ratings, setRatings] = useState({});
    const [activeVoter, setActiveVoter] = useState(0);
    const [view, setView] = useState("rate");
    const [search, setSearch] = useState("");
    const [onlyRated, setOnlyRated] = useState(false);
    const [showSheetModal, setShowSheetModal] = useState(false);
    const [sheetConfig, setSheetConfig] = useState(null); // { apiKey, sheetId }
    const [syncStatus, setSyncStatus] = useState("idle"); // idle | syncing | ok |
    const [syncMsg, setSyncMsg] = useState("");
    const syncTimer = useRef(null);

    const computeScore = useCallback((actId, r = ratings) =>

```

```

    VOTERS.reduce((sum, _, vi) => sum + (r[vi]?.[actId] ?? 0), 0), [ratings]);

const sortedActs = [...ACTS]
  .map(a => ({ ...a, score: computeScore(a.id) }))
  .sort((a, b) => b.score - a.score || a.name.localeCompare(b.name));

const filteredActs = ACTS.filter(a => {
  const q = search.toLowerCase();
  if (q && !a.name.toLowerCase().includes(q) && !a.act.toLowerCase().includes(q)
    if (onlyRated && !VOTERS.some((_, vi) => (ratings[vi]?.[a.id] ?? 0) > 0)) return false;
  return true;
}).map(a => ({ ...a, score: computeScore(a.id) }));

// Debounced sheet sync
const scheduleSync = useCallback((newRatings) => {
  if (!sheetConfig) return;
  clearTimeout(syncTimer.current);
  setSyncStatus("pending");
  syncTimer.current = setTimeout(async () => {
    setSyncStatus("syncing");
    try {
      await saveToSheet(sheetConfig.apiKey, sheetConfig.sheetId, newRatings, (i
        VOTERS.reduce((s, _, vi) => s + (r[vi]?.[id] ?? 0), 0));
      setSyncStatus("ok");
      setSyncMsg("Saved " + new Date().toLocaleTimeString());
    } catch (e) {
      setSyncStatus("error");
      setSyncMsg(e.message || "Sync failed");
    }
  }, 1500);
}, [sheetConfig]);

const handleRate = useCallback((voterId, actId, stars) => {
  setRatings(prev => {
    const next = { ...prev, [voterId]: { ...(prev[voterId] || {}), [actId]: sta
    if (stars === 0) delete next[voterId][actId];
    scheduleSync(next);
    return next;
  });
}, [scheduleSync]);

const handleConnect = async (apiKey, sheetId) => {
  // Try to read first - if empty, initialise; if has data, load it
  let loaded = null;
  try { loaded = await loadFromSheet(apiKey, sheetId); } catch (_) {}
  if (!loaded) {
    await initSheet(apiKey, sheetId);
  }
}

```

```

    loaded = {};
  }
  setSheetConfig({ apiKey, sheetId });
  if (loaded && Object.keys(loaded).length > 0) setRatings(loaded);
  setSyncStatus("ok");
  setSyncMsg("Connected!");
  setShowSheetModal(false);
};

// Poll for remote changes every 15s if connected
useEffect(() => {
  if (!sheetConfig) return;
  const interval = setInterval(async () => {
    if (syncStatus === "syncing" || syncStatus === "pending") return;
    try {
      const remote = await loadFromSheet(sheetConfig.apiKey, sheetConfig.sheetI
      if (remote) setRatings(remote);
    } catch (_) {}
  }, 15000);
  return () => clearInterval(interval);
}, [sheetConfig, syncStatus]);

const ratedCount = ACTS.filter(a => VOTERS.some((_, vi) => (ratings[vi]?.[a.id]

const syncColor = { idle: "rgba(255,255,255,0.25)", pending: "#FFD93D", syncing
const syncIcon = { idle: "o", pending: "o", syncing: "●", ok: "✓", error: "x" }

return (
  <div style={{ minHeight: "100vh", background: "#0c0c18", fontFamily: "'Barlow
    /* Ambient bg */
  <div style={{
    position: "fixed", inset: 0, zIndex: 0, pointerEvents: "none",
    background: `
      radial-gradient(ellipse 60% 40% at 15% 15%, rgba(255,107,157,0.07) 0%,
      radial-gradient(ellipse 50% 50% at 85% 85%, rgba(77,150,255,0.07) 0%, t
      radial-gradient(ellipse 40% 30% at 50% 50%, rgba(255,217,61,0.04) 0%, t
    `,
  }} />

  <div style={{ position: "relative", zIndex: 1, maxWidth: 880, margin: "0 au

    /* — HEADER — */
  <div style={{ textAlign: "center", padding: "30px 0 18px" }}>
    <div style={{ fontSize: 52, marginBottom: -4, filter: "drop-shadow(0 0
    <h1 style={{
      fontFamily: "'Barlow Condensed', sans-serif", fontWeight: 900,
      fontSize: "clamp(44px, 9vw, 78px)", letterSpacing: 3, margin: "4px 0

```



```

        background: "linear-gradient(130deg, #FF6B9D 0%, #FFD93D 45%, #4D96FF
WebkitBackgroundClip: "text", WebkitTextFillColor: "transparent", lin
    }}>
    EUROVISION<br/>2026
</h1>
<div style={{ marginTop: 8, display: "flex", gap: 12, justifyContent: "
    <span style={{ color: "rgba(255,255,255,0.35)", fontSize: 12, lettersS
        Vienna · {ratedCount}/{ACTS.length} rated
    </span>
    {/* Sheet sync indicator */}
    <button
        onClick={() => setShowSheetModal(true)}
        style={{
            display: "flex", alignItems: "center", gap: 5,
            background: "rgba(255,255,255,0.04)", border: `1px solid ${syncCo
            borderRadius: 100, padding: "4px 10px", cursor: "pointer",
            fontSize: 11, color: syncColor, fontWeight: 700, letterSpacing: 0
            transition: "all 0.3s",
        }}
    >
        <span style={{ fontSize: 14 }}>{syncIcon}</span>
        {sheetConfig ? (syncMsg || "Google Sheets") : "Connect Sheets"}
    </button>
</div>
</div>

{/* — VOTER PILLS — */}
<div style={{ display: "flex", gap: 7, flexWrap: "wrap", justifyContent:
    {VOTERS.map((v, i) => {
        const rated = Object.keys(ratings[i] || {}).length;
        return (
            <button key={i} onClick={() => setActiveVoter(i)} style={{
                padding: "7px 16px", borderRadius: 100,
                border: `2px solid ${activeVoter === i ? VOTER_COLORS[i] : "rgba(
                background: activeVoter === i ? `${VOTER_COLORS[i]}1a` : "rgba(25
                color: activeVoter === i ? VOTER_COLORS[i] : "rgba(255,255,255,0.
                fontWeight: 700, fontSize: 13, cursor: "pointer", transition: "al
                letterSpacing: 0.3,
            }}>
                {VOTER_EMOJIS[i]} {v} {rated > 0 && <span style={{ opacity: 0.6,
            </button>
        );
    })}
</div>

{/* — TABS — */}
<div style={{ display: "flex", gap: 0, marginBottom: 18, background: "rgb

```

```

{[["rate", "★ Rate Acts"], ["leaderboard", "🏆 Leaderboard"]].map(([key,
  <button key={key} onClick={() => setView(key)} style={{
    flex: 1, padding: "10px 0", border: "none", borderRadius: 9,
    background: view === key ? "rgba(255,255,255,0.09)" : "none",
    color: view === key ? "#fff" : "rgba(255,255,255,0.35)",
    fontWeight: 700, fontSize: 14, cursor: "pointer", transition: "all
    fontFamily: "'Barlow Condensed', sans-serif", letterSpacing: 1,
  }}>{label}</button>
)
)
</div>

{/* — RATE VIEW — */}
{view === "rate" && (
  <>
    <div style={{ display: "flex", gap: 8, marginBottom: 14 }}>
      <input value={search} onChange={e => setSearch(e.target.value)}
        placeholder="Search country, artist, or song..."
        style={{
          flex: 1, background: "rgba(255,255,255,0.05)", border: "1px solid
          borderRadius: 10, color: "#fff", padding: "10px 14px", fontSize
        }} />
      <button onClick={() => setOnlyRated(p => !p)} style={{
        padding: "10px 14px", borderRadius: 10,
        border: `1px solid ${onlyRated ? "#FFD93D55" : "rgba(255,255,255,
        background: onlyRated ? "rgba(255,217,61,0.1)" : "rgba(255,255,25
        color: onlyRated ? "#FFD93D" : "rgba(255,255,255,0.4)",
        fontSize: 12, fontWeight: 700, cursor: "pointer", whiteSpace: "no
      }}>Rated only</button>
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 9 }}>
      {filteredActs.length === 0 && (
        <div style={{ textAlign: "center", padding: 50, color: "rgba(255,
        )}
        {filteredActs.map(act => {
          const rank = sortedActs.findIndex(a => a.id === act.id);
          return <ActCard key={act.id} act={act} ratings={ratings} onRate={
            activeVoter={activeVoter} score={act.score} rank={rank} total={
          )}
        )}
      </div>
    </>
  )}

{/* — LEADERBOARD VIEW — */}
{view === "leaderboard" && (
  <div>
    {/* Group ranking */}
    <div style={{ marginBottom: 30 }}>

```

```

<h2 style={{ fontFamily: "'Barlow Condensed', sans-serif", fontSize
  🌐 Group Ranking
</h2>
{sortedActs.filter(a => a.score > 0).length === 0 && (
  <div style={{ textAlign: "center", padding: 30, color: "rgba(255,
)}}
<div style={{ display: "flex", flexDirection: "column", gap: 5 }}>
  {sortedActs.filter(a => a.score > 0).map((act, i) => {
    const medalBg = [
      "rgba(255,215,0,0.08)", "rgba(192,192,192,0.06)", "rgba(205,1
    ][i] || "rgba(255,255,255,0.025)";
    const medalBorder = [
      "rgba(255,215,0,0.3)", "rgba(192,192,192,0.2)", "rgba(205,127
    ][i] || "rgba(255,255,255,0.06)";
    return (
      <div key={act.id} style={{
        display: "flex", alignItems: "center", gap: 12,
        background: medalBg, border: `1px solid ${medalBorder}`,
        borderRadius: 10, padding: "9px 14px",
      }}>
        <span style={{ fontSize: i < 3 ? 20 : 13, width: 28, textAl
          {RANK_MEDALS[i]}
        </span>
        <span style={{ fontSize: 22 }}>{act.country}</span>
        <div style={{ flex: 1, minWidth: 0 }}>
          <div style={{ fontWeight: 700, fontSize: 14 }}>{act.name}
          <div style={{ fontSize: 11, color: "rgba(255,255,255,0.35
        </div>
        {/* Per-voter mini stars */}
        <div style={{ display: "flex", gap: 6, flexShrink: 0 }}>
          {VOTERS.map((_, vi) => {
            const s = ratings[vi]?.[act.id] ?? 0;
            return s > 0 ? (
              <div key={vi} title={VOTERS[vi]} style={{ fontSize: 1
                {VOTER_EMOJIS[vi]}{s}★
              </div>
            ) : null;
          })}
        </div>
        <span style={{ fontFamily: "'Barlow Condensed', sans-serif"
          {act.score}
        </span>
      </div>
    );
  })}
</div>
</div>

```

```

    {/* Individual Top 5s */}
    <h2 style={{ fontFamily: "'Barlow Condensed', sans-serif", fontSize:
      🇨🇪 Individual Top 5s
    </h2>
    <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill
      {VOTERS.map((voter, vi) => {
        const vr = ratings[vi] || {};
        const top = ACTS
          .filter(a => (vr[a.id] ?? 0) > 0)
          .sort((a, b) => (vr[b.id] ?? 0) - (vr[a.id] ?? 0))
          .slice(0, 5);
        return (
          <div key={vi} style={{
            background: `${VOTER_COLORS[vi]}0c`,
            border: `1px solid ${VOTER_COLORS[vi]}2a`,
            borderRadius: 12, padding: "12px 14px",
          }}>
            <div style={{ color: VOTER_COLORS[vi], fontWeight: 800, fontS
              {VOTER_EMOJIS[vi]} {voter.toUpperCase()}
            </div>
            {top.length === 0 && <div style={{ color: "rgba(255,255,255,0
              {top.map((act, i) => (
                <div key={act.id} style={{ display: "flex", gap: 8, alignIt
                  <span style={{ fontSize: i < 3 ? 16 : 11, width: 20, text
                    <span style={{ fontSize: 16 }}>{act.country}</span>
                    <span style={{ fontSize: 12, color: "rgba(255,255,255,0.6
                      <span style={{ color: VOTER_COLORS[vi], fontWeight: 700,
                    </div>
                  </div>
                )}}
              </div>
            )};
          )}}
        </div>
      </div>
    )}
  </div>

```

```

{showSheetModal && <SheetModal onConnect={handleConnect} onClose={() => set

```

```

<style>{`
  @import url('https://fonts.googleapis.com/css2?family=Barlow:wght@400;600
  * { box-sizing: border-box; }
  input::placeholder { color: rgba(255,255,255,0.2); }
  ::-webkit-scrollbar { width: 5px; }
  ::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.08); border-ra
  button:hover { opacity: 0.85; }

```

```
`}</style>
</div>
);
}
```